

Minimisation d'une énergie discrète de forme quadratique

On veut résoudre le système d'équations $Ax = b$ où A est une matrice symétrique définie positive. Dans la suite, on ne considère que des systèmes d'équations dans $\mathbb{R}$.

Symétrique : $A^t = A$

pour tout couple de vecteurs x et y : $x^t A y = x^t A^t y = (x^t A^t y)^t = y^t A x$

Définie positive : pour tout vecteur x , on a $x^t A x > 0$

On associe au système d'équations une énergie discrète E définie par

$$E(x) = \frac{1}{2} x^t A x - x^t b$$

Imaginons que l'on puisse faire une petite variation δx sur le vecteur x , l'énergie devient :

$$E(x + \delta x) = E(x) + \frac{1}{2} \delta x^t A x + \frac{1}{2} x^t A \delta x + \frac{1}{2} \delta x^t A \delta x - \delta x^t b$$

Or $\delta x^t A x = x^t A \delta x$, on obtient :

$$E(x + \delta x) = E(x) + \delta x^t (A x - b) + \frac{1}{2} \delta x^t A \delta x$$

$$E(x + \delta x) = E(x) + \delta x^t G(x) + \frac{1}{2} \delta x^t H \delta x$$

Par comparaison, on identifie le vecteur gradient et la matrice hessienne à

$$G(x) \equiv A x - b \text{ et } H \equiv A$$

Le point $x = x_0$ correspond au minimum

$$\text{si } G(x_0) = A x_0 - b = 0 \text{ et}$$

$$\text{si } \delta x^t A \delta x > 0 \text{ pour tout } \delta x \Rightarrow A \text{ matrice définie positive : toutes les valeurs}$$

propres positives

Donc minimiser l'énergie discrète revient à résoudre le système d'équations $Ax = b$

Minimisation de l'énergie discrète selon une direction

On part d'un vecteur x_k et on veut minimiser l'énergie selon une direction p_k .

Autrement dit trouver $x_{k+1} = x_k + \alpha_k p_k$

$$E(x_{k+1}) = E(x_k + \alpha_k p_k) = E(x_k) + \alpha_k p_k^t G(x_k) + \frac{1}{2} \alpha_k^2 p_k^t A p_k$$

$$\frac{\partial E(x_{k+1})}{\partial \alpha_k} = p_k^t G(x_k) + \alpha_k p_k^t A p_k = 0 \Rightarrow \alpha_k = -\frac{p_k^t G(x_k)}{p_k^t A p_k}$$

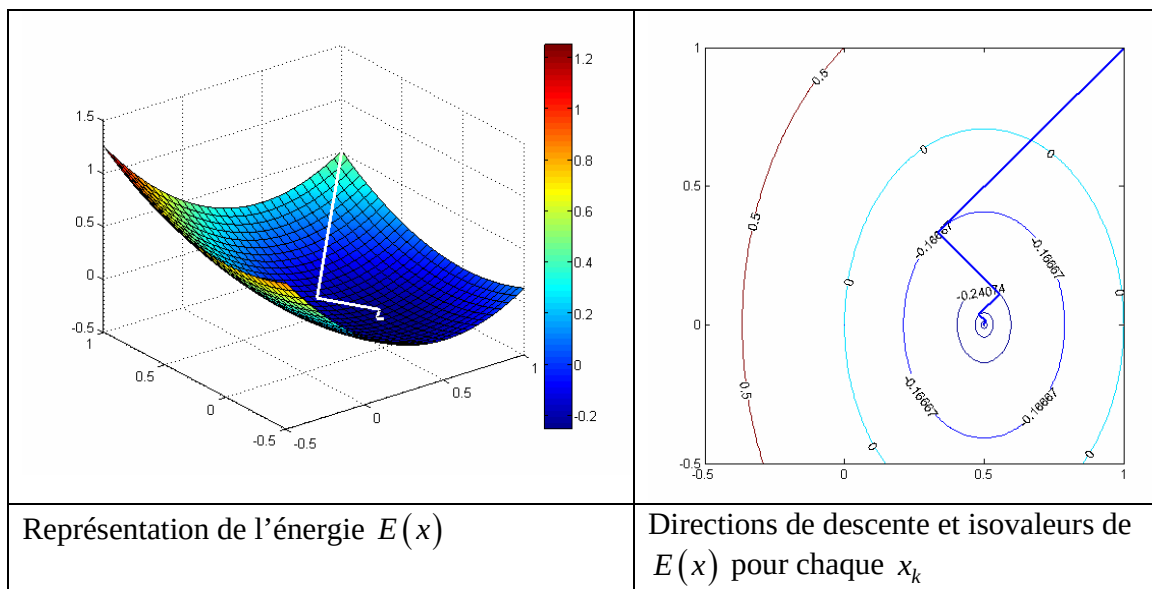
$$\text{En notant } r_k = -G(x_k), \text{ on obtient } \alpha_k = \frac{p_k^t r_k}{p_k^t A p_k}.$$

Minimisation de l'énergie discrète par la méthode du gradient

On choisit $p_k = r_k$ d'où $\alpha_k = \frac{r_k^t r_k}{r_k^t A r_k}$

La procédure itérative de résolution s'écrit : $x_{k+1} = x_k + \frac{r_k^t r_k}{r_k^t A r_k} r_k$

Exemple : résolution de $Ax = b$ avec $A = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$ et $b = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$



Programme 1 : méthode du gradient en matlab

```
% systeme a resoudre
```

```
A=[2 0; 0 1];
```

```
b=[1 0]';
```

```
% point de depart x0
```

```
x=[1 1]';
```

```
% on efface les tableaux
```

```
clear xn yn en;
```

```
% processus de minimisation
```

```
tol=1e-6;
```

```
n=1;
```

```
r=b-A*x;
```

```
while (norm(r)/norm(b) > tol)
```

```
    xn(n)=x(1);
```

```
    yn(n)=x(2);
```

```
    en(n)=0.5*x'*A*x-x'*b;
```

```
    alpha=r'*r/(r'*A*r);
```

```
    x=x+alpha*r;
```

```
    r=b-A*x;
```

```
    n=n+1;
```

```
end;
```

Remarque : la fonction *norm* calcule la norme euclidienne

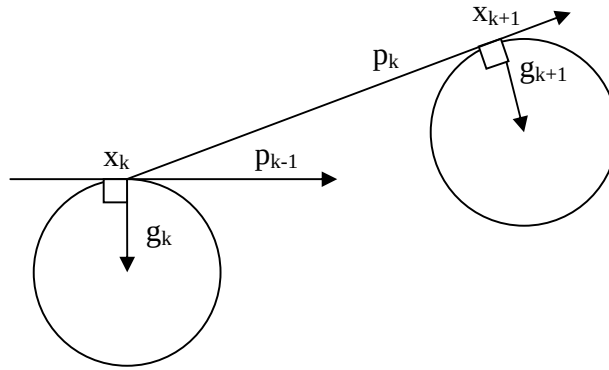
Exercice : montrer en écrivant un petit programme matlab que $r_{k+1}^t r_k = 0$ pour deux itérations successives.

Minimisation par la méthode du gradient conjugué

Dans cette approche, la direction de descente à l'itération k tient compte de la direction de descente à l'itération précédente. On l'écrit comme une combinaison linéaire :

$$p_k = r_k + \beta_{k-1} p_{k-1} \text{ où } r_k = -g_k.$$

et on impose que les directions de descente p_{k-1} et p_k soient conjuguées $p_{k-1}^t A p_k = 0$



Détermination du paramètre β_{k-1}

$$p_{k-1}^t A p_k = 0 \Rightarrow p_{k-1}^t A r_k + \beta_{k-1} p_{k-1}^t A p_{k-1} = 0 \Rightarrow \beta_{k-1} = -\frac{p_{k-1}^t A r_k}{p_{k-1}^t A p_{k-1}}$$

Le point x_{k+1} est déterminé à l'itération k par la relation : $x_{k+1} = x_k + \alpha_k p_k$

$$\Rightarrow \alpha_k = \frac{p_k^t r_k}{p_k^t A p_k}$$

Propriété 1 : par construction, on a $p_k^t r_{k+1} = 0$ ou $r_{k+1}^t p_k = 0$

Propriété 2 : $r_k^t p_k = r_k^t r_k$

Dem. : $r_k^t p_k = r_k^t r_k + \beta_{k-1} r_k^t p_{k-1}$ d'où $r_k^t p_k = r_k^t r_k$ en utilisant la propriété 1

Par conséquent, l'expression du paramètre α_k devient $\alpha_k = \frac{p_k^t r_k}{p_k^t A p_k} = \frac{r_k^t r_k}{p_k^t A p_k}$

Ou encore $\alpha_k p_k^t A p_k = r_k^t r_k$

Propriété 3 : $\alpha_k r_k^t A p_k = \alpha_k p_k^t A p_k = r_k^t r_k$

Dem. : comme $p_k = r_k + \beta_{k-1} p_{k-1}$, on a $\alpha_k r_k^t A p_k = \alpha_k (p_k^t - \beta_{k-1} p_{k-1}^t) A p_k$

Les directions p_{k-1} et p_k étant conjuguées, on obtient $\alpha_k r_k^t A p_k = \alpha_k p_k^t A p_k = r_k^t r_k$

Propriété 4 : le produit scalaire entre deux gradients déterminés aux points x_k et x_{k+1} est nul : $r_k^t r_{k+1} = 0$.

Dem. : par définition $r_{k+1} = b - A x_{k+1} = b - A x_k - \alpha_k A p_k$ d'où $r_{k+1} = r_k - \alpha_k A p_k$

On en déduit que $r_k^t r_{k+1} = r_k^t r_k - \alpha_k r_k^t A p_k = 0$ en utilisant la propriété 3.

Propriété 5 : $\beta_k = \frac{r_{k+1}^t r_{k+1}}{r_k^t r_k}$

Dem. : comme $r_{k+1} = r_k - \alpha_k A p_k$, on a $A p_k = -\frac{r_{k+1} - r_k}{\alpha_k}$.

Par définition $\beta_k = -\frac{p_k^t A r_{k+1}}{p_k^t A p_k}$ et comme A est symétrique, on déduit la relation suivante

en utilisant les propriétés 3 et 4 : $\beta_k = \frac{1}{\alpha_k} \frac{(r_{k+1}^t - r_k^t) r_{k+1}}{p_k^t A p_k} = \frac{(r_{k+1}^t - r_k^t) r_{k+1}}{r_k^t r_k} = \frac{r_{k+1}^t r_{k+1}}{r_k^t r_k}$

Finalement la direction de descente à l'itération k+1 s'exprime en fonction

$$p_{k+1} = r_{k+1} + \beta_k p_k = r_{k+1} - \frac{r_{k+1}^t r_{k+1}}{r_k^t r_k} p_k$$

Algorithme :

Initialisation pour k=0 : $p_0 = r_0$

Tant que $\|r_k\|_2 / \|b\|_2 > tol$

$$\alpha_k = \frac{p_k^t r_k}{p_k^t A p_k} = \frac{r_k^t r_k}{p_k^t A p_k}$$

$$x_{k+1} = x_k + \alpha_k p_k$$

$$\beta_k = -\frac{p_k^t A r_{k+1}}{p_k^t A p_k} = \frac{r_{k+1}^t r_{k+1}}{r_k^t r_k}$$

$$p_{k+1} = r_{k+1} + \beta_k p_k$$

$$k = k + 1$$

Fin boucle

Le processus itératif s'arrête lorsque l'erreur relative $\|r_k\|_2 / \|b\|_2$ devient inférieure à une valeur minimale tol .

Exemple précédent revisité: résolution de $Ax = b$ avec $A = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$ et $b = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

Programme 2 : méthode du gradient conjugué en matlab

```
% systeme a resoudre
```

```
A=[2 0; 0 1];
```

```
b=[1 0]';
```

```
% point de depart x0
```

```
x=[1 1]';
```

```
% on efface les tableaux
```

```
clear xn yn en;
```

```
% tolérance
```

```
tol=1e-6;
```

```
% initialisation
```

```
r=b-A*x;
```

```
p=r;
```

```
n=1;
```

```
xn(n)=x(1);
```

```
yn(n)=x(2);
```

```
en(n)=0.5*x'*A*x-x'*b;
```

```
% processus de minimisation
```

```
while (norm(r)/norm(b) > tol)
```

```
    alpha=p'*r/(p'*A*p);
```

```
    x=x+alpha*p;
```

```
    r=b-A*x;
```

```
    beta=-p'*A*r/(p'*A*p);
```

```
    p=r+beta*p;
```

```
    n=n+1;
```

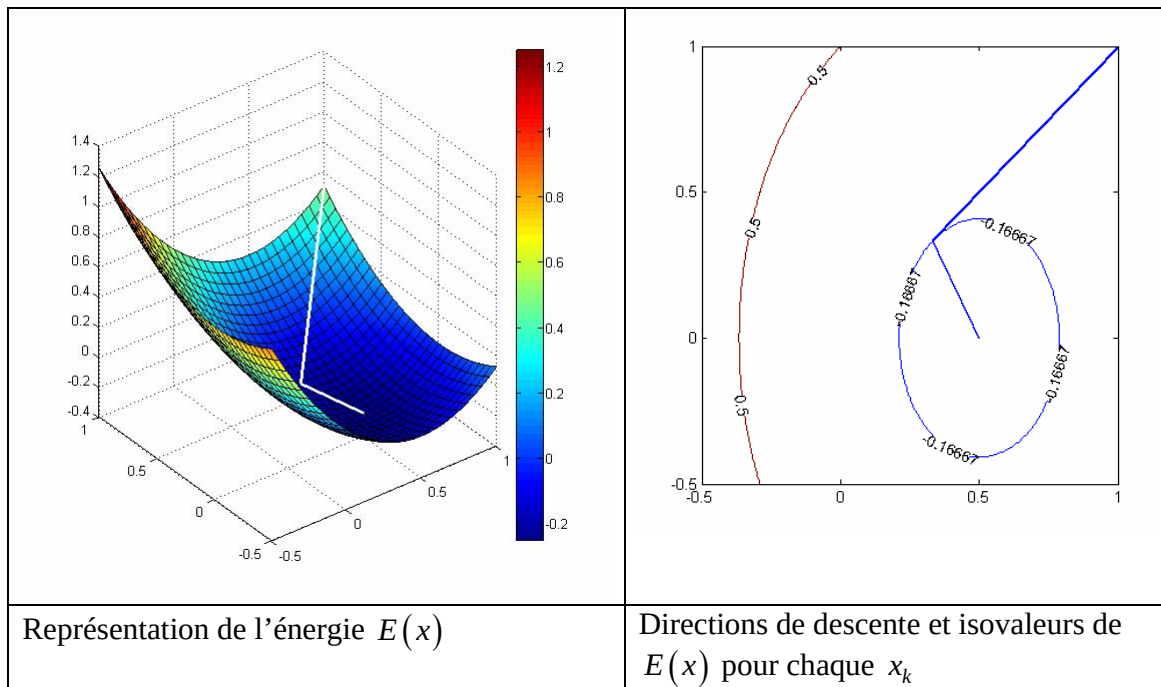
```
    xn(n)=x(1);
```

```
    yn(n)=x(2);
```

```
    en(n)=0.5*x'*A*x-x'*b;
```

```
end;
```

```
disp(['nombre d'iterations'  
num2str(n-1)]);
```



Programme 2 optimisé : méthode du gradient conjugué en matlab

```
% systeme a resoudre
A=[2 0; 0 1];
b=[1 0]';

% point de depart x0
x=[1 1]';

% on efface les tableaux
clear xn yn en;

% tolérance
tol=1e-6;

% initialisation
r=b-A*x;
p=-r;
rho1=r'*r;

n=1;
xn(n)=x(1);
yn(n)=x(2);

en(n)=0.5*x'*A*x-x'*b;

% processus de minimisation
while (norm(r)/norm(b) > tol)
    alpha=rho1/(p'*A*p)
    x=x+alpha*p
    r=b-A*x;
    rho=r'*r;
    beta=rho/rho1;
    p=r+beta*p;
    rho1=rho;

    n=n+1;
    xn(n)=x(1);
    yn(n)=x(2);
    en(n)=0.5*x'*A*x-x'*b;
end;

disp(['nombre d'iterations'
num2str(n-1)]);
```

La représentation graphique est réalisée sous la forme par le code suivant :

```
hold off
[x,y] = meshgrid([-0.5:.05:1], [-0.5:.05:1]);
Z =0.5*(A(1,1)*x.*x+A(1,2)*x.*y+A(2,1)*y.*x+A(2,2)*y.*y)-b(1)*x-b(2)*y;
surf(x,y,Z,Z, 'FaceColor','interp')
hold on;
plot3(xn, yn, en, 'w-', 'LineWidth',2)
colorbar

figure
[x,y] = meshgrid([-0.5:.01:1], [-0.5:.01:1]);
Z =0.5*(A(1,1)*x.*x+A(1,2)*x.*y+A(2,1)*y.*x+A(2,2)*y.*y)-b(1)*x-b(2)*y;
[C,h] =contour(x, y, Z, en)
set(h,'ShowText','on','TextStep',get(h,'LevelStep')*2)
colormap jet
hold on;
plot(xn, yn, 'b-', 'LineWidth',2)
```

Application de la méthode du gradient conjugué sur des systèmes de grande taille

La méthode du gradient conjugué est bien adaptée pour résoudre des systèmes d'équations linéaires où la matrice associée A , obligatoirement symétrique définie positive, contient peu d'éléments. On dit qu'elle est creuse.

```
% random avec la meme graine
rand ('seed',12563);
randn('seed',12563);

% taille du systeme
N=400; density=0.01;
A = sprandn(N,N,density);

% pour rendre A symetrique
A=0.5*(A+A');

% pour rendre A definie positive
D=sum(abs(A(:,1:N)));
for i=1:N
    A(i,i)=D(i)-A(i,i);
end;

disp(['nb d'elts non nuls '
num2str(nnz(A))]);
disp(['conditionnement de A '
num2str(max(eig(A))/min(eig(A)))
]);

% construction de b
b=rand(N, 1);

% point de depart x0
x=rand(N, 1);

% on efface les tableaux
clear xn en;

tol=1e-6; % tolérance

% initialisation
r=b-A*x;
p=r;
rho1=r'*r;

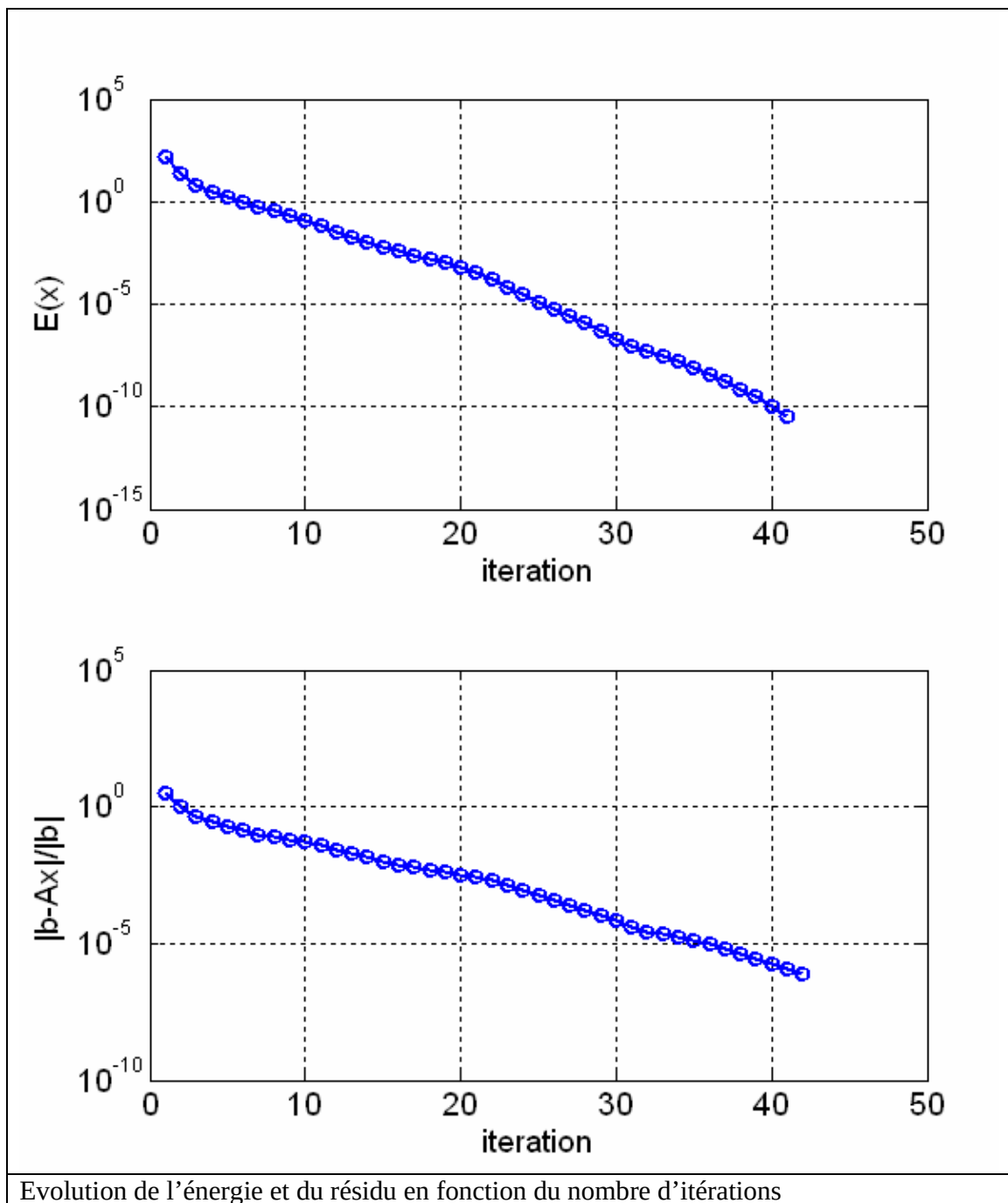
n=1;
xn(:, n)=x;
en(n)=0.5*x'*A*x-x'*b;

% processus de minimisation
while (norm(r)/norm(b) > tol)
    alpha=rho1/(p'*A*p);
    x=x+alpha*p;
    r=b-A*x;
    rho=r'*r;
    beta=rho/rho1;
    p=r+beta*p;
    rho1=rho;

    n=n+1;
    xn(:, n)=x;
    en(n)=0.5*x'*A*x-x'*b;
end;

disp(['nombre d'iterations'
num2str(n-1)]);

% trace en echelle log10
ef=en(end); % valeur finale
semilogy(abs(en-ef));
grid on
```



Préconditionnement

Le préconditionnement est une technique pour le conditionnement d'une matrice. Supposons que C soit une matrice symétrique positive qui approxime la matrice A et qui est facile à inverser. La résolution de $Ax = b$ se fait de manière indirecte en résolvant :

$$C^{-1}Ax = C^{-1}b.$$

Si $\text{Cond}(C^{-1}A) \ll \text{Cond}(A)$ ou si le spectre des valeurs propres de $C^{-1}A$ est moins étendu que celui de A alors la résolution du système modifié s'effectuera plus rapidement que celle du système original. Le problème est que $C^{-1}A$ n'est pas forcément symétrique même si C et A le sont.

Pour contourner cette difficulté, il suffit de décomposer C en un produit de matrices $C = EE^t$ par une méthode de Cholesky.

Les matrices $C^{-1}A$ et $E^{-1}AE^{-t}$ (où $E^{-t} := (E^{-1})^t$) ont les mêmes valeurs propres. Si v est un vecteur propre de $C^{-1}A$ avec la valeur propre λ alors $E^t v$ est vecteur propre de $E^{-1}AE^{-t}$ avec la même valeur propre λ :

$$(E^{-1}AE^{-t})E^t v = (E^{-1}A)v = E^t E^{-t} E^{-1}Av = E^t C^{-1}Av = \lambda E^t v$$

Le système $C^{-1}Ax = C^{-1}b$ peut être transformé en un nouveau système $E^{-1}AE^{-t}\hat{x} = E^{-1}b$ en définissant $\hat{x} = E^t x$.

Comme la matrice $E^{-1}AE^{-t}$ a le même spectre de valeurs propres que $C^{-1}A$, elle a le même conditionnement. De plus, elle reste dans tous les cas symétrique et définie positive. La résolution de ce système peut donc être réalisée par la méthode du gradient conjugué.

L'application de l'algorithme du gradient conjugué vu précédemment donne à condition de définir $\hat{r}_k = E^{-1}b - E^{-1}AE^{-t}\hat{x}_k = -E^{-1}r_k$

Algorithme du gradient conjugué non transformé :

Initialisation pour $k=0$: $\hat{p}_0 = \hat{r}_0$

Tant que $\|\hat{r}_k\|_2 / \|E^{-1}b\|_2 > tol$

$$\alpha_k = \frac{\hat{r}_k^t \hat{r}_k}{\hat{p}_k^t E^{-1}AE^{-t} \hat{p}_k}$$
$$\hat{x}_{k+1} = \hat{x}_k + \alpha_k \hat{p}_k$$
$$\beta_k = \frac{\hat{r}_{k+1}^t \hat{r}_{k+1}}{\hat{r}_k^t \hat{r}_k}$$

$$\hat{p}_{k+1} = \hat{r}_{k+1} + \beta_k \hat{p}_k$$

$$k = k + 1$$

Fin boucle

L'inconvénient majeur est qu'il faut calculer E^{-1} .

L'astuce consiste à définir en plus de $\hat{r}_k = E^{-1} r_k$, $\hat{p}_k = E^t p_k$.

Algorithme du gradient conjugué préconditionné :

Initialisation pour $k=0$: $p_0 = E^{-t} \hat{p}_0 = E^{-t} \hat{r}_0 = E^{-t} E^{-1} r_0$ d'où $p_0 = C^{-1} r_0$

Tant que $\|r_k\|_2 / \|b\|_2 > tol$

$$\alpha_k = \frac{r_k^t C^{-1} r_k}{p_k^t A p_k}$$

$$x_{k+1} = x_k + \alpha_k p_k$$

$$\beta_k = \frac{r_{k+1}^t C^{-1} r_{k+1}}{r_k^t C^{-1} r_k}$$

$$p_{k+1} = C^{-1} r_{k+1} + \beta_k p_k$$

$$k = k + 1$$

Fin boucle

Rem1 : $x_{k+1} = E^{-t} \hat{x}_{k+1} = E^{-t} \hat{x}_k + \alpha_k E^{-t} \hat{p}_k$ d'où $x_{k+1} = x_k + \alpha_k p_k$

Rem2 : $p_{k+1} = E^{-t} \hat{p}_{k+1} = E^{-t} \hat{r}_{k+1} + \beta_k E^{-t} \hat{p}_k \Rightarrow p_{k+1} = E^{-t} E^{-1} r_{k+1} + \beta_k p_k$
 $\Rightarrow p_{k+1} = C^{-1} r_{k+1} + \beta_k p_k$

Programme 3 : méthode du gradient conjugué préconditionné en matlab

```
% random avec le meme graine
rand ('seed',12563);
randn('seed',12563);

% taille du systeme
N=400;
density=0.01;
A = sprandn(N,N,density);

% pour rendre A symetrique
A=0.5*(A+A');

% HACK pour rendre A definie
positive
D=sum(abs(A(:,1:N)));
for i=1:N
    A(i,i)=D(i)-A(i,i);
end;

% Preconditionneur
M=diag(1./diag(A));

disp (['nb d''elts non nuls '
num2str(nnz(A))]);
disp (['conditionnement '
num2str(max(eig(M*A))/min(eig(M*A)
))]);

%M=inv(A);

% construction de b
b=rand(N, 1);

% point de depart x0
x=rand(N, 1);

% on efface les tableaux

clear xn en;

% tolérance
tol=1e-6;

% initialisation
r=b-A*x;
p=M*r;
rho1=p'*r;

n=1;
xn(:, n)=x;
en(n)=0.5*x'*A*x-x'*b;

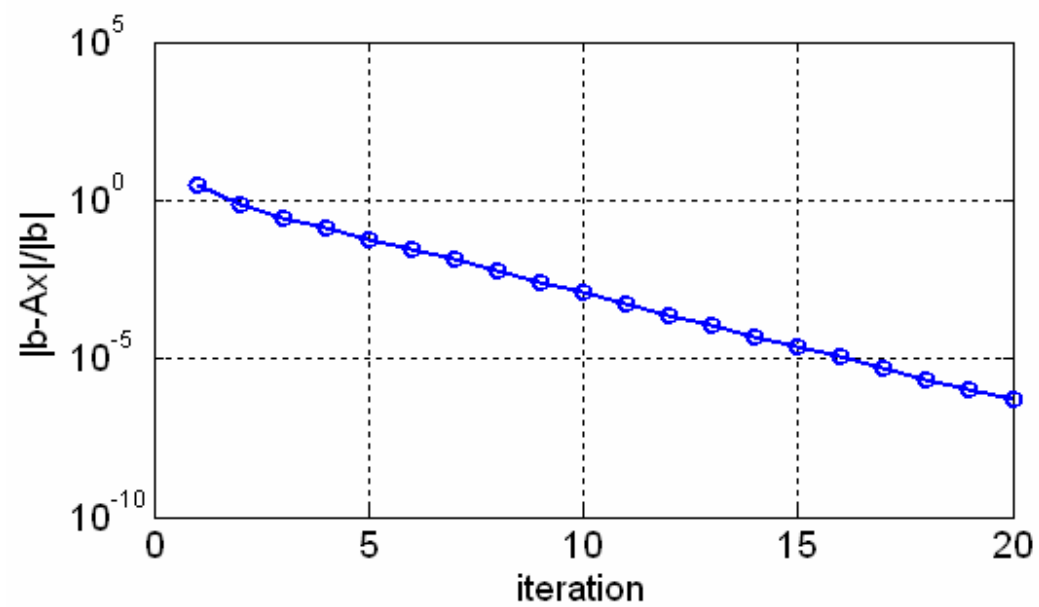
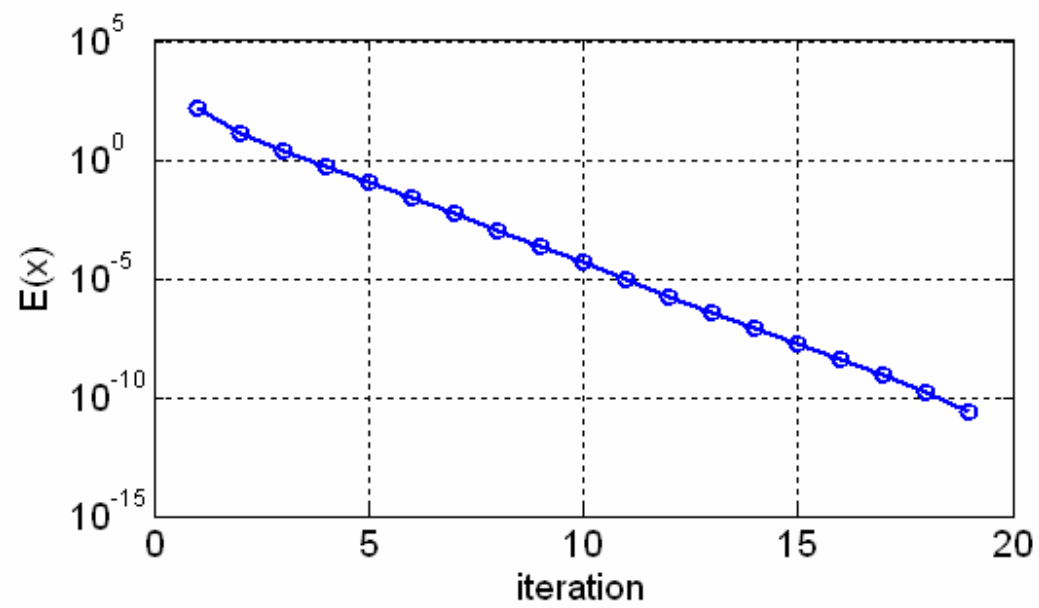
% processus de minimisation
while (norm(r)/norm(b) > tol)
    alpha=rho1/(p'*A*p);
    x=x+alpha*p;
    r=b-A*x;
    rho=r'*M*r;
    beta=rho/rho1;
    p=M*r+beta*p;
    rho1=rho;

    n=n+1;
    xn(:, n)=x;
    en(n)=0.5*x'*A*x-x'*b;
end;

disp (['nombre d''iterations'
num2str(n-1)]);

ef=en(end); % valeur finale

% trace en echelle log10
semilogy(abs(en-ef));
grid on
```



Evolution de l'énergie et du résidu en fonction du nombre d'itérations